Session ID (Re-) Creation

It is a good idea to regenerate session IDs periodically, especially if the session contains sensitive data, and cycling IDs is a good way to invalidate stolen sessions. Storing expiry timestamps in the session itself allows you to detect when to expire a session ID and generate a new one. Storing an "obsolete" flag is another good idea – this way you can detect attempts to access an obsolete session in addition to knowing when to regenerate the session ID. Here's some code that will achieve that:

```
session_start();

$now = time();
$_SESSION['expire'] = $_SESSION['expire'] ?? $now + 30;

if ($now > $_SESSION['expire']) {
    $_SESSION['obsolete'] = true;
    session_regenerate_id();
    unset($_SESSION['obsolete']);
    $_SESSION['expire'] = $now + 30;
    print "Session expired! Regenerated with new ID: " . session_
    id() . "\n";
} else {
    print "Session still active with ID: " . session_id() . "\n";
}
```

This simple code will set a 30-second expiry timestamp inside the session if one does not already exist (i.e., when a user visits with no `PHPSESSID` cookie set). On each request, the `$_SESSION['expiry']` value is tested against the current time; when the server time exceeds the expiry, we add a `$_SESSION['obsolete']` boolean to the session data, then call `session_regenerate_id()` to retain the current session data but with a new ID. Obviously this is going to result in a cookie header, so it must be called before output begins (remember to use output buffering if this causes problems for script logic). Since we're now dealing with a "new" session, we unset the obsolete flag and add a new expiry stamp. Let's put it to the test, using curl's cookie support (the `-b` and `-c` options specify a file to read and write cookie data):

```
$ curl -b ~/.curl_cookies -c ~/.curl_cookies http://localhost:8080/
index.php
Session still active with ID: 54a9a1f51c7842c0a58d0b971cb463d1
```